FAST LOOKUP WITH HASHTABLES

Problem Solving with Computers-II



Amazon IP Tracking Problem

Imagine you're a software engineer at Amazon. It's Prime Day. Your team is responsible for tracking unique visitors to the site.

Here's your problem: 200 million users are hitting the site today. Every single page request, you need to check if this IP address has visited before. If not, add it to your set of known visitors.

Your code runs on every. single. request. It needs to be fast!

What data structure do you use?

U: Universe of Keys

192 (8 bits)	168 (8 bits)	1 (8 bits)	6 (8 bits)

An IP is 32 bits → 4.3B (2^32) possible IPs

O(1) worst case….at a cost

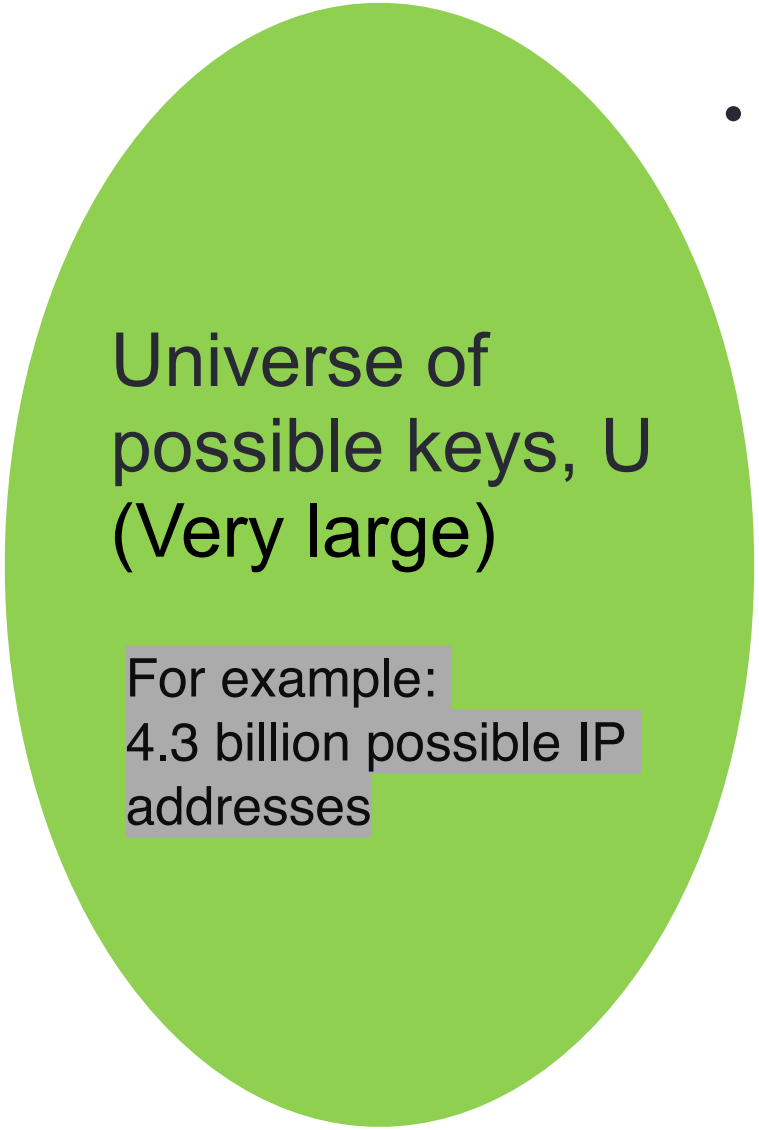
192 (8 bits)	168 (8 bits)	1 (8 bits)	6 (8 bits)

Use a very large vector with 2^{32} possible spots

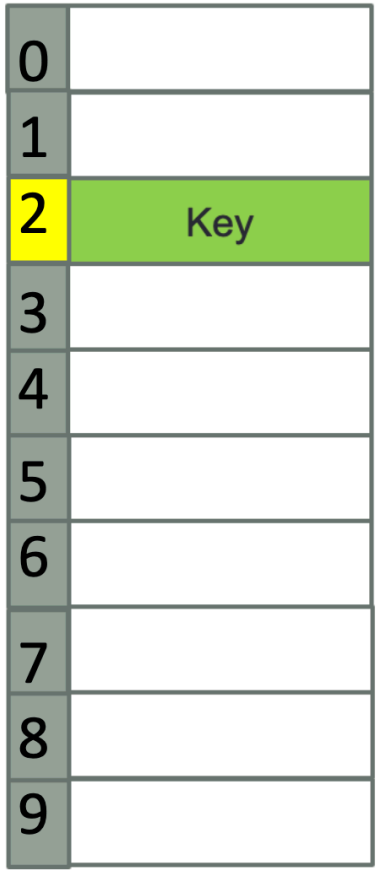
First approach: Use key value (IP) as index in a 2^{32} sized vector

But you're only tracking 200 million: about 5% of the space
So, this method wastes space.

Setup for hash tables



- Keep track of evolving set S whose size is much less than the universe of all possible keys
- For example, 200M unique users (~ 5% of all possible IPs)

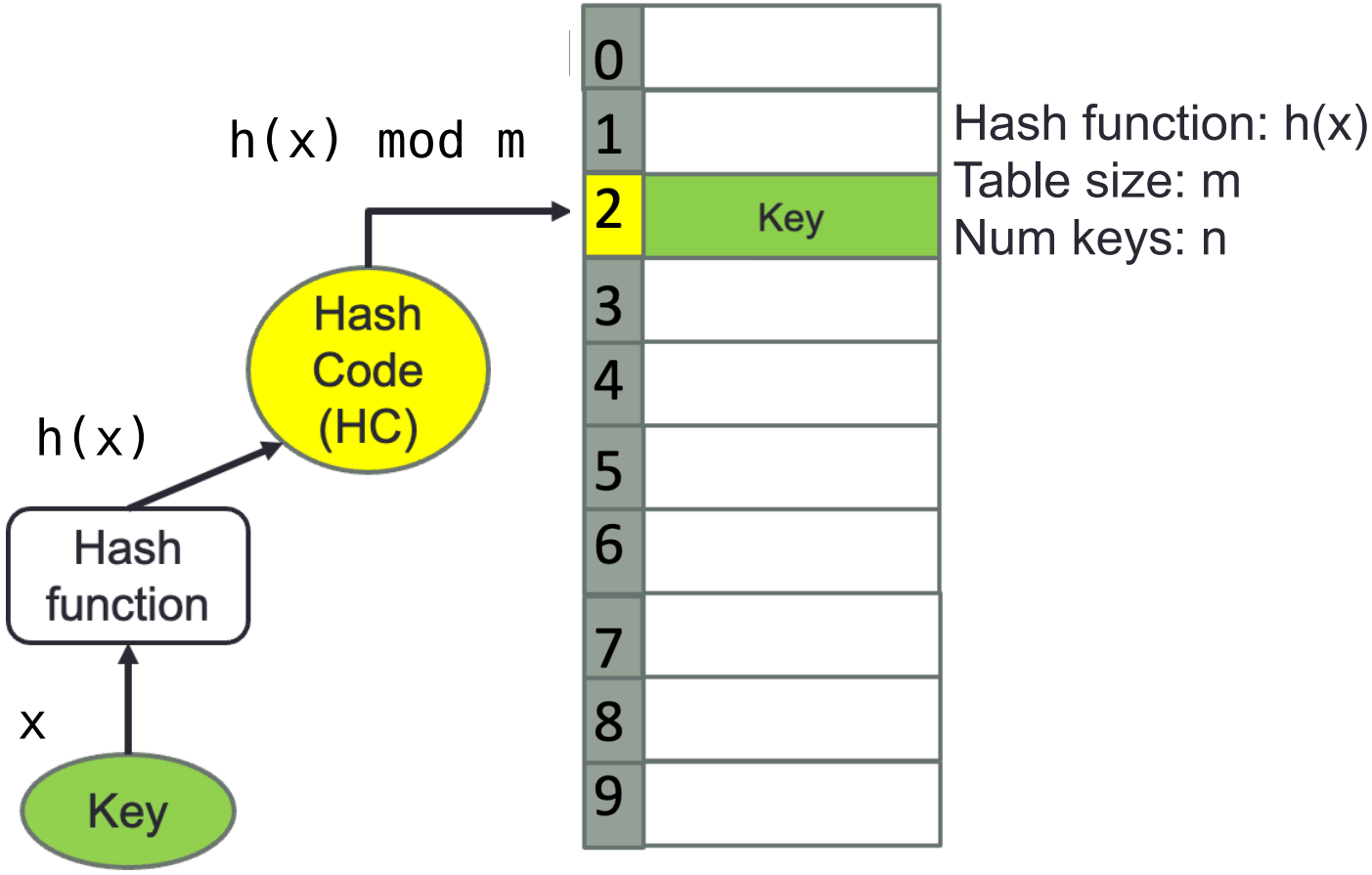


Setup for hash tables

- Keep track of evolving set S whose size is much less than the universe of all possible keys
- For example, 200M unique users (~ 5% of all possible IPs)

Universe of possible keys, U (Very large)

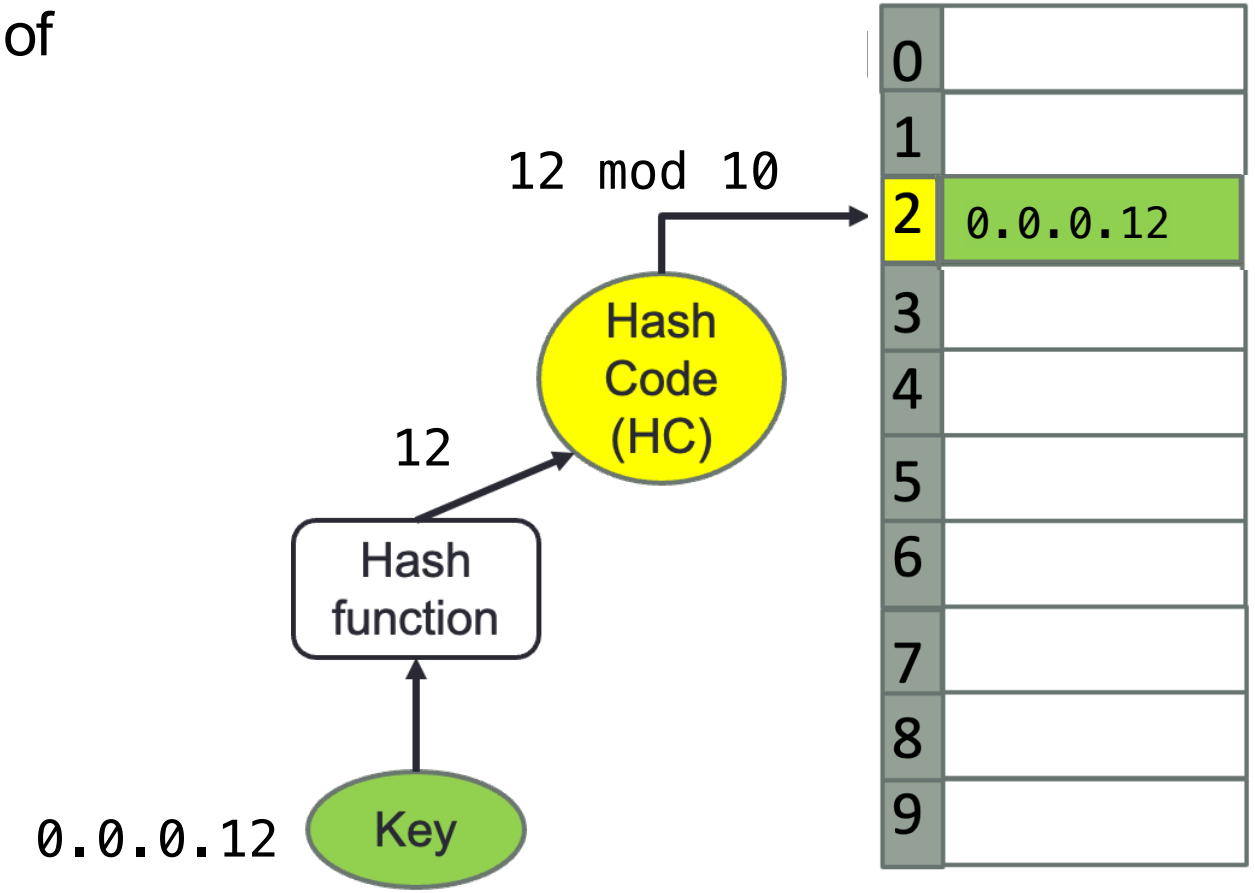
For example:
4.3 billion possible IP addresses



Structure of a hash table

- Keys stored in buckets (vector)
- Keys used to compute index of position in vector

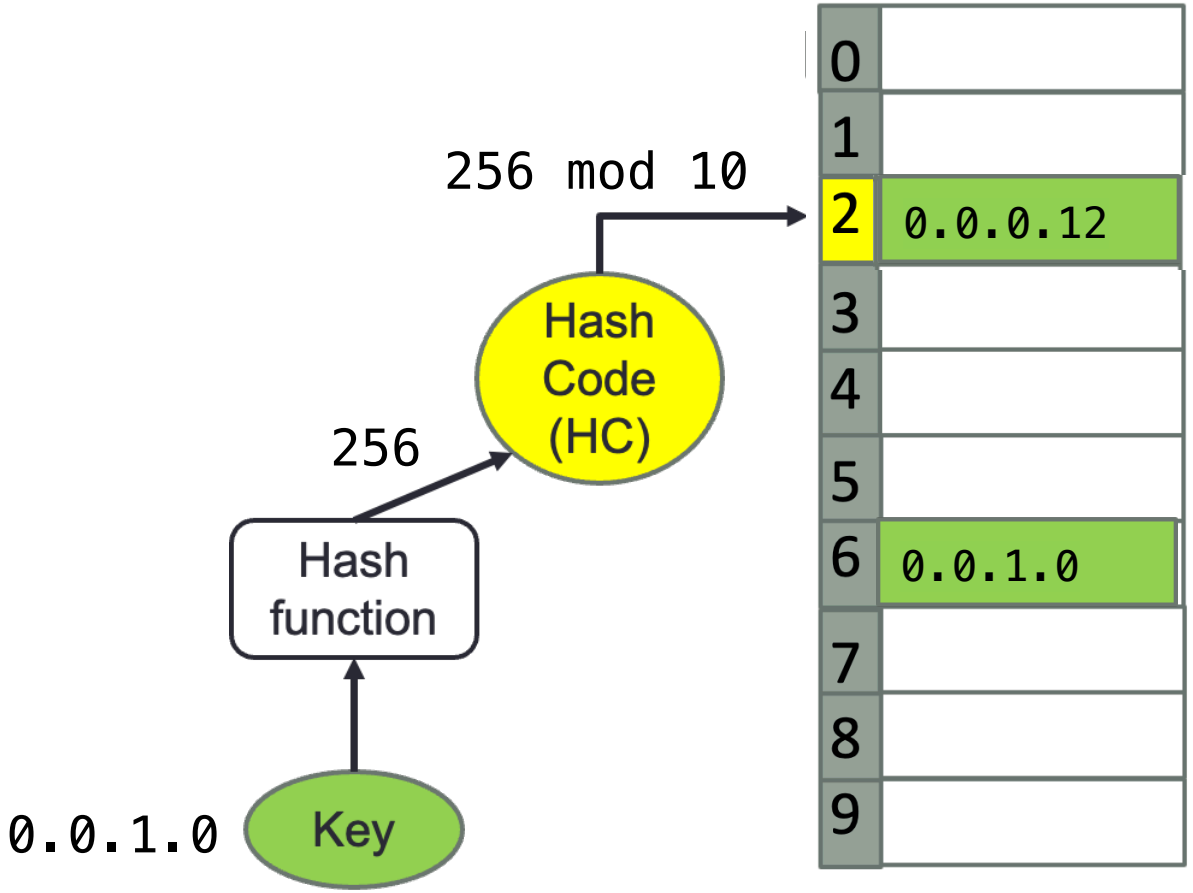
Hash function: h(x)
Table size: m
Num keys: n



Structure of a hash table

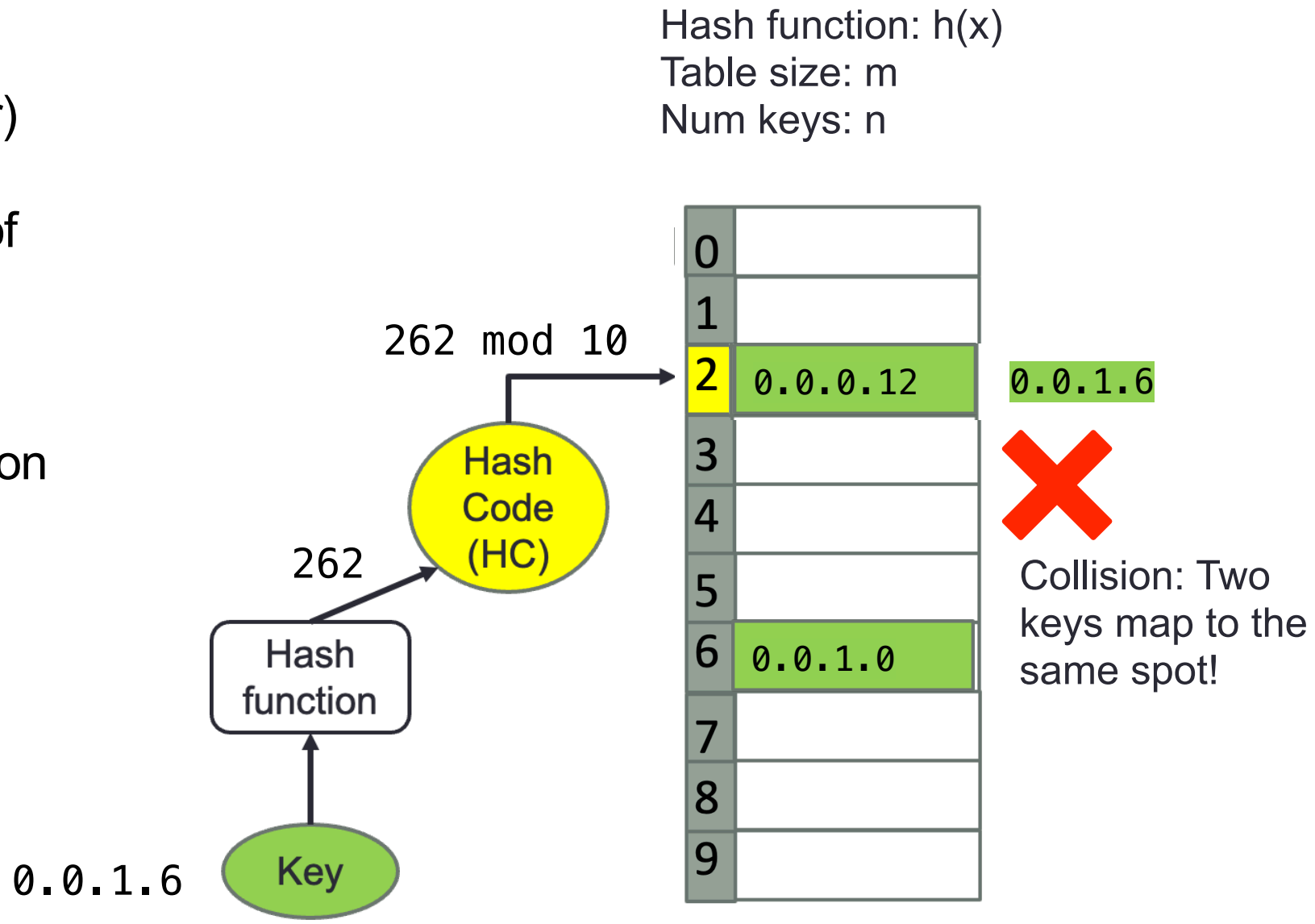
- Keys stored in buckets (vector)
- Keys used to compute index of position in vector

Hash function: h(x)
Table size: m
Num keys: n



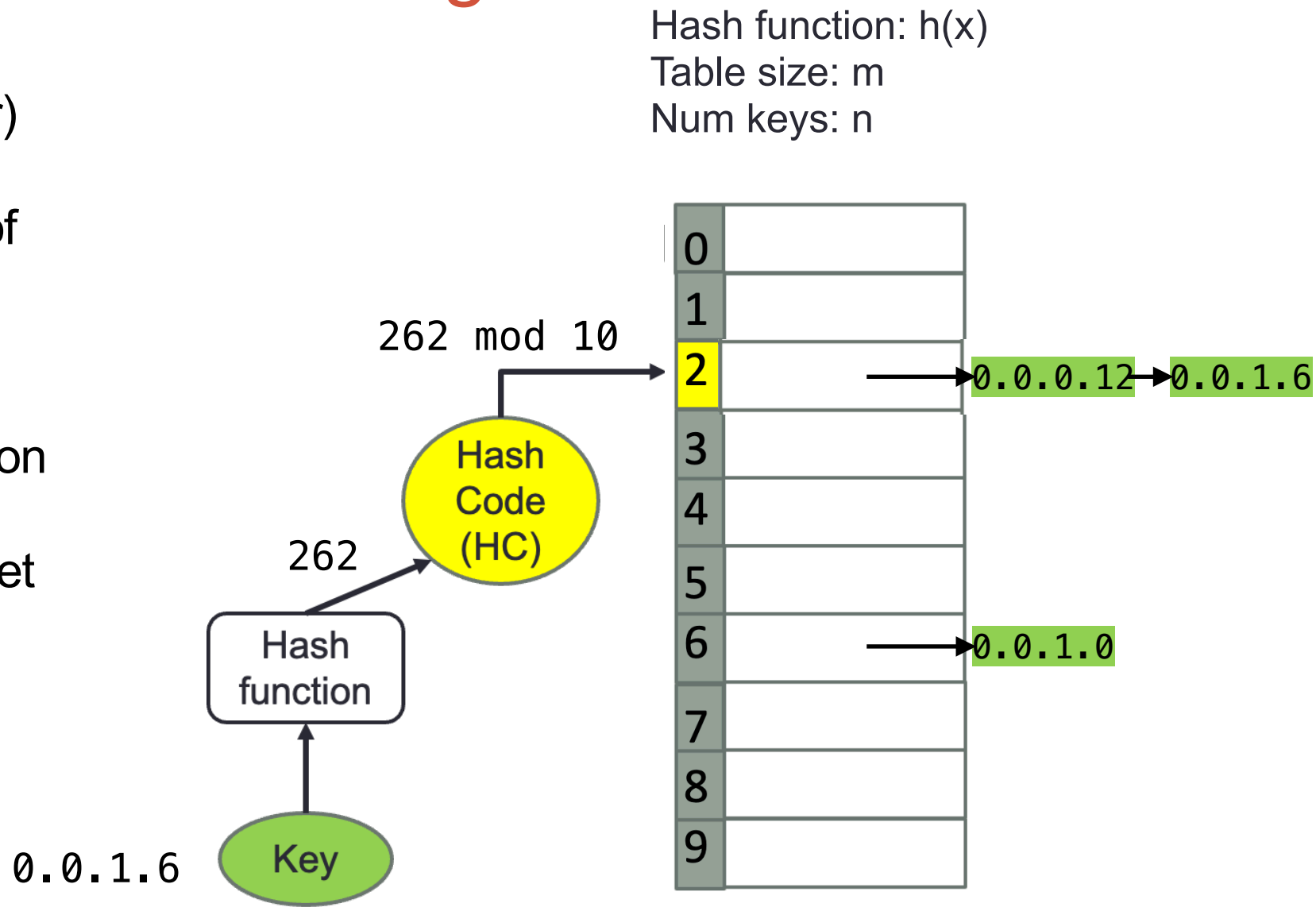
Structure of a hash table

- Keys stored in buckets (vector)
- Keys used to compute index of position in vector
- When two keys map to the same bucket we have a collision



Hash table with separate chaining

- Keys stored in buckets (vector)
- Keys used to compute index of position in vector
- When two keys map to the same bucket we have a collision
- Separate-chaining: each bucket store a list of keys



O(1) to search…. Really?

What does O(1) average case really look like?

What's the failure mode?

Code Demo: iptracking.cpp

```
// Hash table with separate chaining
class unordered_set{
 public:
  void insert(Key k) {
      int idx = hash(k) % m;
      buckets[idx].push_back(k);  // O(1) insert at tail
  }

  int count(Key k) {
      int idx = hash(k) % m;
      // Linear search through buckets[idx]
      for (auto& item : buckets[idx]) {
          if (item == k) return 1;
      }
      return 0;
  }

 private:
  vector<list<KeyType>> buckets;  // or vector<forward_list<>>
};
```

Activity 1: Hash Function Practice (5 min)

Task: Given a hash table of size 10 and $h(x) = x \bmod 10$:

- Insert keys: 23, 45, 12, 78, 32, 55, 92
- Identify which keys collide
- Count how many buckets are occupied
- What is the load factor α = n/m?

Average case analysis

Setup:

- n keys, m buckets
- Load factor: α = n/m
- Assumptions

Derive: Average time to search for a key = ?

iClicker: Determining the hash table size

You want to place 1000 elements in a hash table, using chained hashing.
You'd like a successful search to examine 2 keys on average.
How big does the table need to be?

A. 500

B. 1000

C. 250

D. 125

E. 2000

Two flavors of hash tables in C++ STL

```
#include <unordered_set>  // unique keys only
#include <unordered_map>  // key-value pairs
```

	unordered_set<T>	unordered_map<K,V>
Stores	Keys only	Key-value pairs
Use case	"Have I seen this?"	"What's the value for this key?"
Example	Tracking unique IPs	Counting visits per IP

Web traffic analysis

You're back at your job at Amazon. Your boss hands you a log of IP addresses from today's traffic and asks:

1. How many unique visitors did we have?
2. How many times did each IP visit?
3. Who are our top K most frequent visitors?

Open traffic_activity.cpp

STL unordered_set

```
unordered_set<string> seen_ips;

// Insert - O(1) average
seen_ips.insert("192.168.1.1");
seen_ips.insert("10.0.0.5");
seen_ips.insert("192.168.1.1");  // ignored, already exists

// Check membership - O(1) average
if (seen_ips.count("192.168.1.1")) {  // returns 0 or 1
    cout << "Seen before!";
}

// Size
cout << seen_ips.size();  // 2 (not 3!)

// Remove - O(1) average
seen_ips.erase("10.0.0.5");
```

STL unordered_map

```
unordered_map<string, int> visit_count;

// Insert / Update - O(1) average
visit_count["192.168.1.1"] = 1;       // insert
visit_count["192.168.1.1"]++;         // update: now 2
visit_count[“10.0.0.5"]++; //auto-creates with 0, then increments to 1

// Lookup - O(1) average
cout << visit_count["192.168.1.1"];  // 2

// Check if key exists (without creating it!)
if (visit_count.count("8.8.8.8")) {
     cout << visit_count["8.8.8.8"];
}
// Iterate all pairs
for (auto& [ip, count] : visit_count) {
     cout << ip << ": " << count << "\n";
 }
```

When to use a hash table, when not to?

Mapping Unique Identifiers to Values Hash tables are great for associative arrays, dictionaries, and maps

Fast Key-Value Access When you frequently need to retrieve values by a unique key (e.g., looking up user by ID).

Datasets with Unordered Data If you have a dataset and don't require data to be stored in a particular order. Other ordered structures like BST or linked lists, need O(logn) or O(n) time for lookups

Frequent Insertions and Deletions

Since hash tables handle these operations in average O(1) time, they're ideal for use cases where data is constantly added or removed,

Example: caching, session management, or real-time tracking.

Wrap up

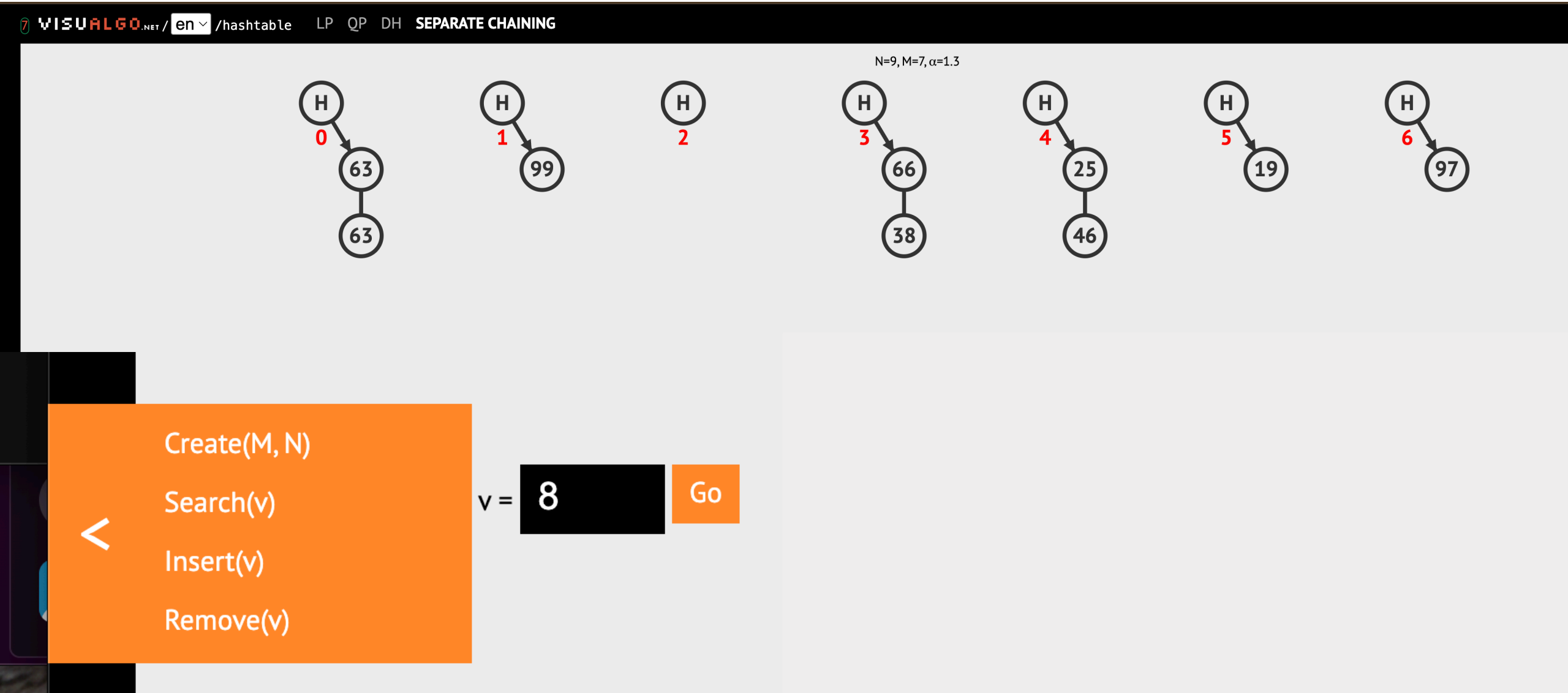
Quiz 3 on Monday of next week

Topics include: BSTs, stacks, queues

Lab 02, PA01

Hash table visualization

https://visualgo.net/en/hashtable



References

Professor Subhash Suri's CS 130A handout on hash tables:

https://sites.cs.ucsb.edu/~suri/cs130a/Hashing.pdf